MCP using spring ai | even tools calling manager:
We send as one of the methods to the chatClient to perform some 3rd party api call maybe or some one more backend service, only structure same with mcp so basically tool_definition -> provide llm about tool functionality, tool_param-> provide the structure and schema for input data, tool_calling_param is the json/rpc that contains the collected data or call data[name,args] like how we have same structure in .calltools() then tool calling manager-> so what happens here receives json call form the model then underlying function or api gets executed and the result that we get has to be serialized and sent back to llm, tool calling manager is important because it's similar structure here again, the flow is u send something to llm-> back to our spring boot app which has spring ai dependencies, mcp client-> mcp server under the bridge communication through MCProtocol, so mcp client is nothing but when ai system sending input req from llm -> it goes to mcp client from client, backend exposing to tools is mcp server-> transport[we have 2 ways one for http sse way for remote calls, stdio for ai agents]=> basically we have app.yaml for both that we need to enable spring ai mco client stdio enabled:true for spring ai mcp server stdio enabled:true then spring.ai.mcp.client.stdio.sever-configurations[put in one servers.json fror personal project we have github repo with available mcp servers][one way of making our application work, as mcp server the common properties that are very imp when using stdio way -> spring.main.web-application-type:none[not webmvc],for webmvc apps even dependency should be spring ai starter mcp client and server webmvc [in app yaml we provide streamable http connections:url name all these properties sometimes name of server and url where boot app hosted which acts as client], this client can be created and connected to either http/stdio using mcpclient.using(transform).sync().calltool() or mcpsyncclient().calltool()[same structure as [name,args] name of tools and input args for that tool, diff is configuration custom or pre, one can be extended for interceptors,logging, routing and so on], we need toolcallback provider or mcp function callback which is used like to get[function/api that gets executed as per required tool and o/p generated] and sent it during this.chatClient Autowiring adding defaultToolCallback(toolcallback provider here)
toolcallbackprovider=ToolCallback.from(your tool instance) similar to how we sent tools() to chatClient but autowiring has to be done a bit diff here like which toolback provider we need, after that Also very important about @mcptool and @mcptoolparam, understand
server based annotations
@MCPTools, @MCPResource, @MCPPrompt, @MCPComplete
client based annotations
@MCPProgress, @MCPElicitation,@MCPSampling,@MCPLogging
as soon as it finds mcptool it picks up all annotations added to class -> since spring makes them into beans and ready for configuration, now after all this one output will be generated and sent to llm -> beautify it more/ add more context with server output and send it back to client.

one more option to have webflux - the non blocking way configuration mcps-> spring ai starter mcp client webflux and starter mcp server webflux same properties used-> name,type,version and toolcallback.enabled, also add annotation.scanner enabled.
Everything other than application.yaml are same for this too.[once go through spring ai documentation].

==========
==========
========== Tool Routing Strategies:

Tool routing is about decision-making before execution, not execution itself.
Types of Routing
-> LLM-Based Routing
LLM chooses tool based on:
Tool definition (description)
Tool params (schema)

This is what .callTools() does and Works for simple systems
Hard to control in enterprise systems

->Rule-Based Routing **important
if (intent == PAYMENT) -> paymentTool
if (intent == WEATHER) -> weatherTool
Used when: we need predictability
Compliance/security check matters

-> Hybrid Routing (REAL WORLD)
go 1: Rule-based filter (allowed tools)
go 2: LLM chooses within that subset

for ex:
User in finance domain -> restrict tools to finance related only
Then LLM picks specific one
This is what most production systems do.

-> Dynamic Routing (Advanced)
Based on:
Context
Previous tool outputs
User session state
for ex:
If transaction failed -> route to retry tool
If suspicious -> route to fraud tool

==========
==========
==========
====
=> Multi-Tool Orchestration

tool1 + tool2 = combine result sequentially
Thatâ ™s linear orchestration, but real systems go beyond that.
Types of Orchestration
=> Linear (Sequential)
Tool A -> Tool B -> Final Response
Example:
Get user -> get transactions -> summarize

=> Parallel (Independent)
Tool A
        -> Merge -> LLM
Tool B
Example:
Weather API+News API

=> The point about avoiding unnecessary calls is correct here:
Good tool descriptions help LLM avoid useless parallel calls like if u provide only input to one tool why make a route to second tool where that input doesn't work.

=> Conditional (Branching)
if (fraudDetected)
   -> Fraud Tool
else
   -> Payment Tool
Combination: This is where routing + orchestration merge

=> Iterative / Looping
Tool -> Result -> LLM -> Tool -> Result -> LLM.
for ex:

Debugging and Research
LLM keeps deciding next step dynamically

=>Aggregation Pattern
Multiple tools -> Normalize ->Combine -> LLM
for ex:
Fetch cashback offers from multiple banks
Combine+rank

[User Prompt]
      ->
[Routing Layer] -> decides from all available tools
      ->
[Orchestration Layer] -> defines execution flow
(seq/parallel/conditional/loop/aggregators)
      ->
[Execution Layer]
      ->
[Interceptors] -> logging, auth, retry
      ->
[Tool Execution]
      ->
[LLM Response Enhancement]

interceptors also we need to worry because cross cutting concerns for every tool call like input/output allowed to AI, providing api keys before calling service etc.
Retry->If tool fails -> retry 3 times
Rate Limiting->Prevent abuse
==========
==========
==========